KNEEBENCH REPORT · RUN #1

unsloth/Meta-Llama-3.1-8B-Instruct

NVIDIA H100 NVL 94GB (as entered) · vLLM 0.29.0 · chat assistant workload · 2026-10-22 10:14 · done

Setup

Model	unsloth/Meta-Llama-3.1-8B-Instruct, served as llama-8b
GPU	NVIDIA H100 NVL 94GB (as entered)
Server	vLLM 0.29.0, context window 8,192 tokens
KV cache	542,400 tokens in 16-token blocks, same precision as the model, prefix caching on
Scheduler limits	max_num_seqs 1,024 (the default for this vLLM and GPU); about 8,192 tokens per step (the default for this vLLM and GPU)
Traffic	chat assistant, 2,000 tokens in and 500 out
Targets	500 ms to the first token and 50 ms per token, for 99 of 100 requests
Tested from	macOS, Apple silicon, 279 ms round trip to the server
What ran	single-request baseline, concurrency sweep (1 to 256 users), request-rate sweep, request-size sweep, long-prompt interference, prompt reuse, abandoned requests; 47 points, kneebench 0.1.0.dev0, AIPerf 0.13.0

Executive summary

- For your chat assistant traffic (2,000 tokens in, 500 out), this server comfortably handles about 8 users at your target of 50 ms per token and 500 ms to the first token for 99 out of 100 requests. It breaks just above that because time to first token crosses your target, not because memory is full: the KV cache was only 3.5% used there.
- Plan for about 3.58 requests per second: 20% below the 4.47 that met your targets with users arriving at random.
- First change to try: lower the sequence cap (`--max-num-seqs 126`). max_num_seqs is 1,024 (the default) while the cache sustains about 199 of your requests, and the server is already evicting requests.
- From where this test ran, the network adds 279 ms to every first token (more than the server itself takes), so run kneebench closer to the server for first-token numbers.

Scorecard

BASELINE VS PHYSICS	CAPACITY UNDER YOUR SLO	HEADROOM	MEMORY SAFETY	TEST RELIABILITY
72% of memory bandwidth	8 users (928 tok/s) within your targets	Plan for 3.58 req/s to keep 20% for bursts	Evicting requests from 256 users	2 problems change the numbers

What needs attention

With the cap above what memory holds, overload turns into silent evictions instead of a visible queue. A lower cap makes extra requests wait their turn, which you can see and plan for.

Lower the sequence cap

```
--max-num-seqs 126
```

Expected The preemption counter stays at 0 and extra requests wait in the queue (vllm:num_requests_waiting) instead of being evicted.

Risk Peak throughput may dip slightly; queued requests wait longer for their first token.

Verify Rerun the same sweep; preemptions should stay at 0 and the tail latency should stop spiking.

WHAT WE MEASURED

542,400 tokens of cache, handed out in 16-token blocks, divided by 2,710 tokens per finished request gives 199. 95 of 100 requests in the sweep finished within 2,710 tokens, including the chat template. Keeping 15% spare leaves 169. Your per-token target is crossed at about 127 users.

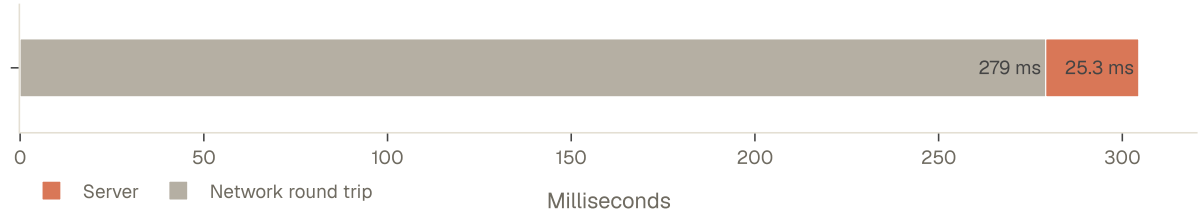
WHY IT HAPPENS

vLLM admits up to max_num_seqs requests at once. Sizing it for a long request, not an average one, with some spare, keeps every admitted request's cache in memory even when several long ones arrive together; lowering it further to where the latency target breaks keeps admitted users fast.

Learn more: Lesson 42: Tune GPU memory and KV capacity, Knob 3: how many requests you let in

(https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/42-tune-gpu-memory-kv-capacity.md#5-knob-3-how-many-requests-you-let-in)

Where the 304 ms to the first token goes, from this location



First-token numbers here mostly measure the network. Time per token is still trustworthy, because tokens stream back-to-back once started.

Run kneebench next to the server, on a machine that keeps up

Expected Time to first token drops by about the round trip (279 ms), and the server's own time to first token becomes measurable.

Risk None. kneebench runs anywhere Python does, such as a pod or VM in the same region.

Verify Rerun from there; this check should turn green.

WHAT WE MEASURED

A request to /health, which the server answers instantly, took 279 ms round trip over a warm connection (new connections: TCP 289 ms, TLS 294 ms). What users at this location see: TTFT *(time to first token, how long a user waits before the answer starts appearing)* 304 ms.

WHY IT HAPPENS

Every request crosses the network before the server starts working, and the first token crosses back. kneebench measures that round trip on its own so it isn't blamed on the server. A client far from the server, or too slow to keep up, ends up benchmarking itself. *Like timing a pizza delivery to judge the oven: the drive dominates.*

Learn more: Lesson 25: Sweep concurrency and request rate, How sweeps quietly go wrong (https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/25-sweep-concurrency-request-rate.md#6-how-sweeps-quietly-go-wrong)

99 out of 100 requests must meet the target; the slowest ones are what users remember. A dashboard of medians would call this load fine.

WHAT WE MEASURED

Median and P99 TTFT at 16 users.

WHY IT HAPPENS

Under load, a few requests land behind long prompts or wait for memory, so the tail grows faster than the median. Judging by medians lets the tail blow through the target unnoticed.

Learn more: Lesson 25: Sweep concurrency and request rate, How sweeps quietly go wrong (https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/25-sweep-concurrency-request-rate.md#6-how-sweeps-quietly-go-wrong)

An FP8 cache is one flag (`--kv-cache-dtype fp8`) and doubles how many requests fit. In the large study the lesson cites, FP8 weights and activations were essentially lossless and speed up both decode and prefill where supported; INT8 weights and activations lost about 1 to 3%; 4-bit weight-only kept about 99% with a good method and suits small batches, its gain shrinking as batches grow. Every one changes the numbers the model computes with: check quality on your own tasks before shipping.

Store the KV cache in FP8

```
--kv-cache-dtype fp8
```

Expected About twice the cache tokens, so about twice as many requests fit before evictions.

Risk The cache holds slightly less exact numbers; quality usually holds, but check it on your own tasks before shipping.

Verify The cache size on /metrics doubles; rerun and check that evictions stop at the same load.

WHAT WE MEASURED

Not measured: projected from the served 16-bit model on the NVIDIA H100 NVL 94GB. Weights shrink with their bits and the freed memory goes to the cache. Decode time follows the roofline at 8 users, scaled from your measured time per token. Prefill only gets faster when the math itself runs in fewer bits.

FORMAT	WEIGHTS	REQUESTS THAT FIT	DECODE AT 1 USER	DECODE AT LOAD	PREFILL
As served	16.1 GB	216	1.0×	1.0×	1.0×
FP8 KV cache	16.1 GB	432	1.0×	1.1×	1.0×
FP8 weights and activations	8.0 GB	240	2.0×	1.8×	2.0×
4-bit weights only	4.1 GB	252	3.7×	2.8×	1.0×

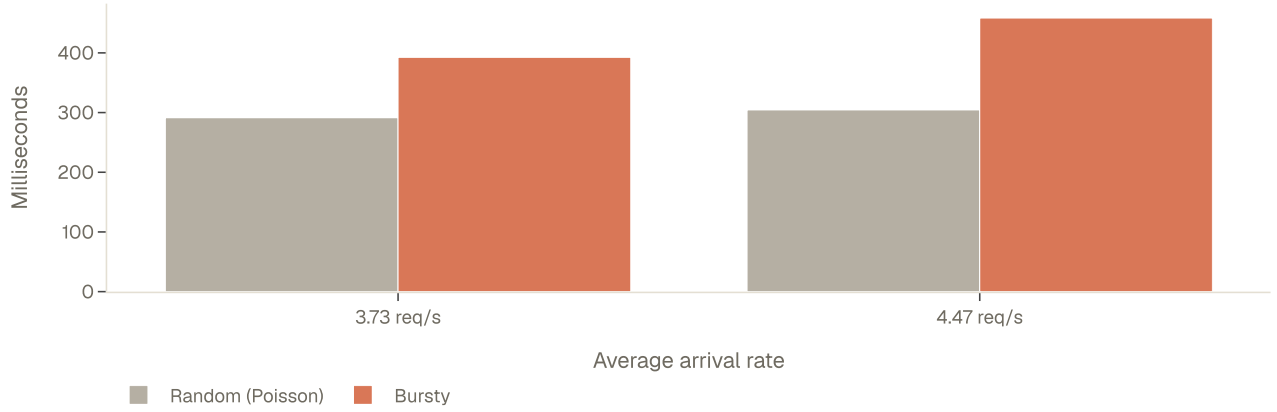
WHY IT HAPPENS

Decode reads every weight and the cache each step, so fewer bytes make it faster. Prefill is math, so it only speeds up when the tensor cores compute in the smaller format. The estimates leave out unpacking costs, so they are upper bounds, closest at small batches.

Learn more: Lesson 50: Precision, memory and hardware support, Two ways to use low precision

(https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/50-precision-memory-hardware-support.md#5-two-ways-to-use-low-precision)

P99 time to first token: steady vs bursty arrivals



Plan capacity with headroom for bursts, or smooth them before they reach the server.

Keep more headroom, or queue requests in front of the server

Expected Burst peaks stay below capacity, so the tail meets the target.

Risk Lower average utilization of the GPU.

Verify Rerun with the rate sweep; bursty and steady results move closer.

WHAT WE MEASURED

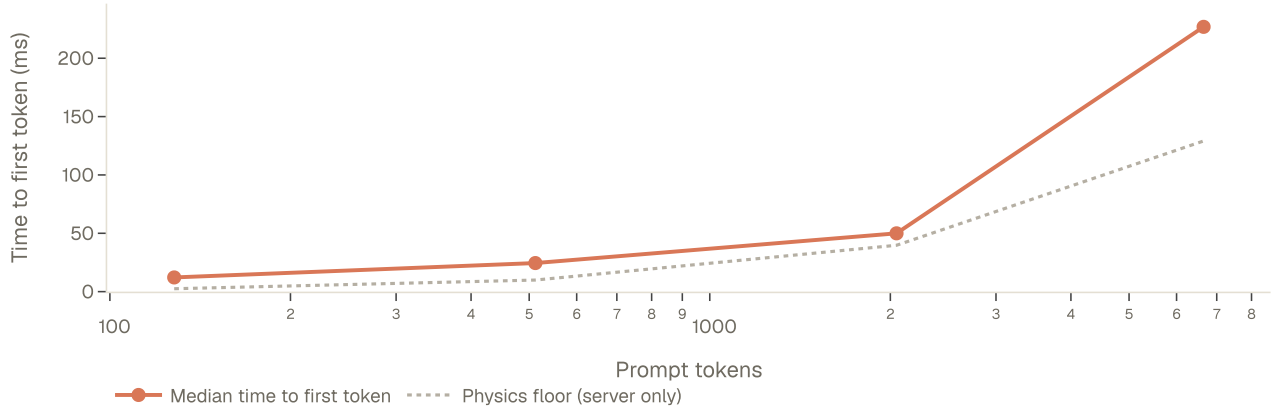
The same average rates sent twice: with random (Poisson) gaps, and with clustered gaps (gamma, burstiness 0.5).

WHY IT HAPPENS

Bursts briefly push the arrival rate above what the server can start, so requests queue even though the average looks fine. The tail pays first.

Learn more: Lesson 25: Sweep concurrency and request rate, Running both with vLLM (https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/25-sweep-concurrency-request-rate.md#5-running-both-with-vllm)

One user: first-token time as the prompt grows



Very long prompts cost more than their length suggests. Trim context, or expect much slower first tokens for them.

WHAT WE MEASURED

One user at a time, prompts from 128 to 6,656 tokens with 256-token answers. The per-token cost is the slope of the line, so the network round trip drops out. Physics allows about 19 µs per prompt token; measured is 19 µs.

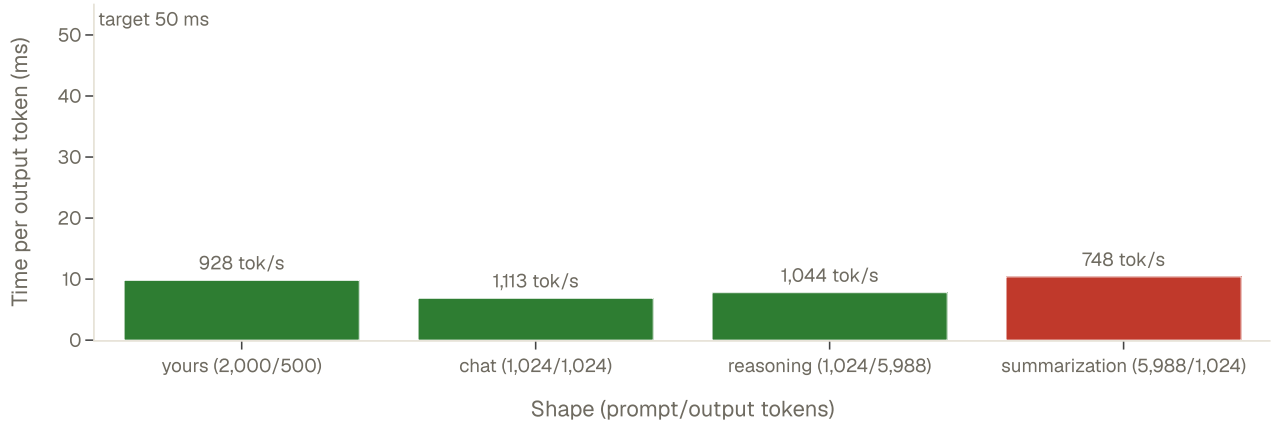
PROMPT TOKENS	TTFT P50	TTFT TAIL
128	12.2 ms	14.4 ms
512	24.5 ms	28.9 ms
2,048	50 ms	59.8 ms
6,656	227 ms	260 ms

WHY IT HAPPENS

The server reads the whole prompt in one pass, called prefill *(the step that reads the whole prompt at once, limited by GPU math)*, limited by compute, so the work grows with prompt length. Attention compares every token with every earlier one, which grows with length squared and bends the line up for long prompts.

Learn more: Lesson 26: Sweep prompt and output length, Sweep the prompt (https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/26-sweep-prompt-output-length.md#2-sweep-the-prompt-output-fixed-at-256)

Every request size at 8 users (green: meets targets)



Settings that work for your traffic fail for other sizes. If you serve mixed traffic, plan for the worst common size or split it across pools.

WHAT WE MEASURED

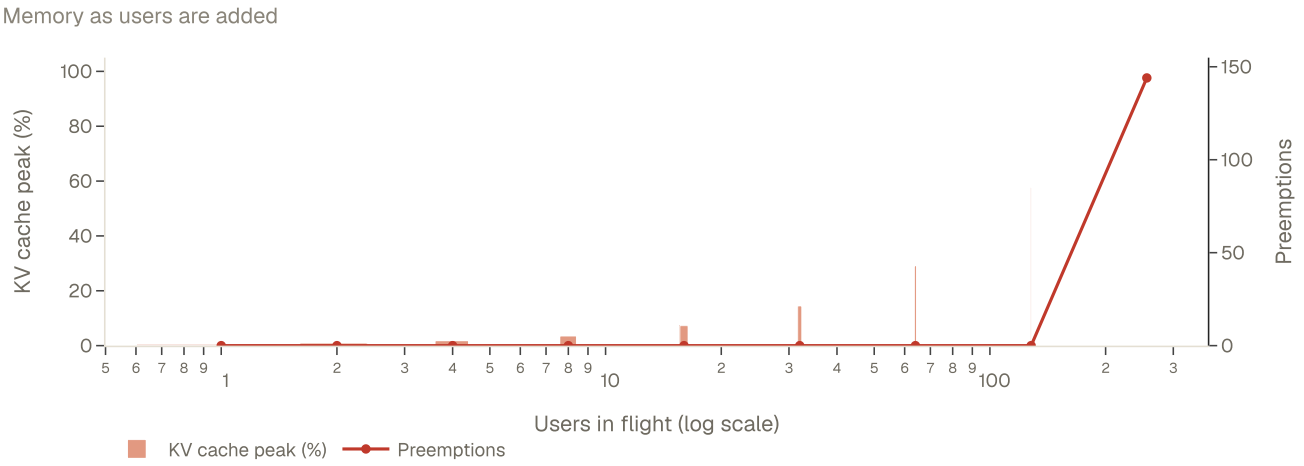
Each standard request size and yours, run separately at the operating point. They are never averaged: long requests would hide inside the mean.

SHAPE	PROMPT	OUTPUT	OUTPUT TOKENS/S	TTFT TAIL	TPOT TAIL	PEAK CACHE	FAILED	MEETS TARGETS
yours	2,000	500	928	500 ms	9.8 ms	3.5%	0%	yes
chat	1,024	1,024	1,113	232 ms	6.9 ms	2.8%	0%	yes
reasoning	1,024	5,988	1,044	246 ms	7.8 ms	9.2%	0%	yes
summarization	5,988	1,024	748	1.79 s	10.4 ms	10%	0%	no

WHY IT HAPPENS

Prompt length sets first-token time and cache size; output length sets answer time and how long each request holds memory. The same GPU fits many short chats but far fewer long documents.

Learn more: Lesson 26: Sweep prompt and output length, The shape map (https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/26-sweep-prompt-output-length.md#6-the-shape-map)



The server let in more requests than memory can hold to the end, then threw some out to make room. Nobody gets an error, but evicted users see long freezes, and recomputing them slows everyone else.

WHAT WE MEASURED

At 256 users the KV cache peaked at 100% and the preemption counter rose by 144. P99 per-token latency went from 50.3 ms to 111 ms.

WHY IT HAPPENS

A request's cache starts small and grows with every token it writes. vLLM admits up to max_num_seqs requests; when the cache hits 100% it evicts one, discards its cache, and later recomputes the prompt plus everything it had generated. The only trace is a counter in /metrics. *Like a restaurant seating every party at once and then asking some to leave mid-meal and come back later.*

Learn more: Lesson 41: Preemption, recomputation and KV pressure, Thrashing (https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/41-preemption-recomputation-kv-pressure.md#4-thrashing)

At those rates the server never saw the load the report describes, so their results are too optimistic.

Run kneebench next to the server, on a machine that keeps up

Expected Sent rates match the requested ones.

Risk None.

Verify This check turns green.

WHAT WE MEASURED

Requests actually started per second, compared with the requested rate.

WHY IT HAPPENS

A sender that can't keep up (CPU, connections or network) quietly benchmarks itself.

Learn more: Lesson 25: Sweep concurrency and request rate, How sweeps quietly go wrong (https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/25-sweep-concurrency-request-rate.md#6-how-sweeps-quietly-go-wrong)

Try these next

Change your server

Settings for your vLLM server or deployment, most important first. Change one at a time and rerun, so you can see what each did.

1 **Lower the sequence cap** W06

```
--max-num-seqs 126
```

Why max_num_seqs is 1,024 (the default) while the cache sustains about 199 of your requests, and the server is already evicting requests.

Expected The preemption counter stays at 0 and extra requests wait in the queue (vllm:num_requests_waiting) instead of being evicted.

Risk Peak throughput may dip slightly; queued requests wait longer for their first token.

2 **Store the KV cache in FP8** F02

```
--kv-cache-dtype fp8
```

Why The cache is the limit (requests were evicted): an FP8 cache would hold 432 requests of 2,000 in and 500 out instead of 216.

Expected About twice the cache tokens, so about twice as many requests fit before evictions.

Risk The cache holds slightly less exact numbers; quality usually holds, but check it on your own tasks before shipping.

3 **Keep more headroom, or queue requests in front of the server** R04

Why Bursts break your target at a rate that steady random traffic meets: P99 time to first token goes from 304 ms to 458 ms (1.5×) when the same traffic arrives in bursts.

Expected Burst peaks stay below capacity, so the tail meets the target.

Risk Lower average utilization of the GPU.

Make the test more reliable

These don't touch your server. They make the next run's numbers more trustworthy.

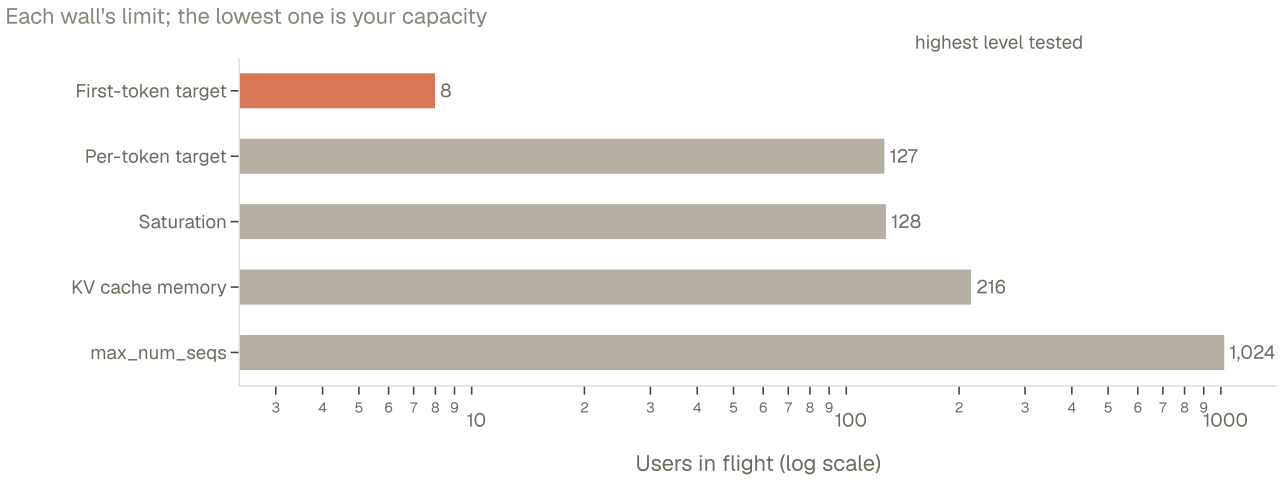
1 **Run kneebench next to the server, on a machine that keeps up** H03, R03

Why Almost all of the time to first token is network: a 279 ms round trip against 304 ms in total. The server's own time to first token can't be estimated reliably from here.

Expected Time to first token drops by about the round trip (279 ms), and the server's own time to first token becomes measurable.

Risk None. kneebench runs anywhere Python does, such as a pod or VM in the same region.

How much load it takes



Whichever wall comes first is the limit; the others don't matter until it moves. Each traffic shape has its own walls.

WHAT WE MEASURED

The walls, in users in flight: first-token target 8; per-token latency target 127; saturation 128; memory (cache ÷ request length) 216; max_num_seqs 1,024.

WHY IT HAPPENS

Every wall gives a maximum number of users for this shape: the latency targets, saturation, memory (cache size divided by each request's final length) and the server's own cap. The smallest one is the capacity.

Learn more: Lesson 27: Find saturation, SLO failure and OOM boundaries, The operating envelope (https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/27-find-saturation-slo-oom-boundaries.md#5-the-operating-envelope)

Long prompts fill the step budget, so new requests wait for their turn to be read. A bigger budget starts them sooner but makes each step longer, which slows every answer streaming at the time. At your 50 ms per-token target, a step should carry at most about 2,688 prompt tokens; the long-prompt interference test shows whether the current budget breaks that.

WHAT WE MEASURED

vLLM admits a request only if all three allow it: a free slot under max_num_seqs (1,024, the default), room in the per-step token budget (about 8,192, the default for your vLLM and GPU), and free cache blocks. The table shows, for each level where requests queued, what the server reported. Percentiles of step size are estimates from its histogram buckets.

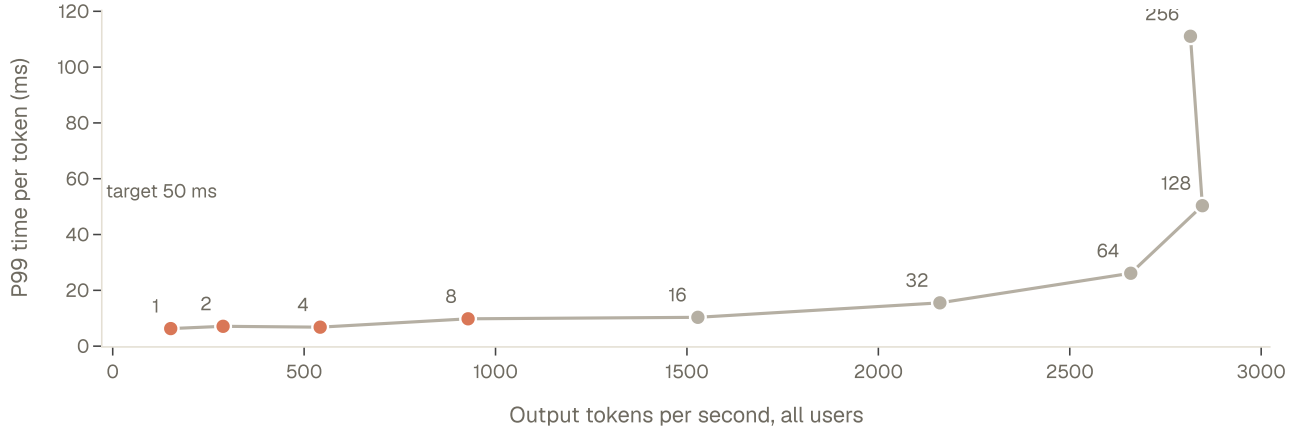
USERS	LIMIT	RUNNING (MAX)	WAITING (MAX)	STEP TOKENS P99	CACHE PEAK	QUEUE P99
16	none full	16	3	2,814	7.3%	83.2 ms
32	none full	32	17	3,592	15%	1.03 s
64	token budget	64	44	7,269	29%	3.59 s
128	token budget	128	101	8,192	58%	8.23 s
256	KV cache	256	203	8,644	100%	15.4 s

WHY IT HAPPENS

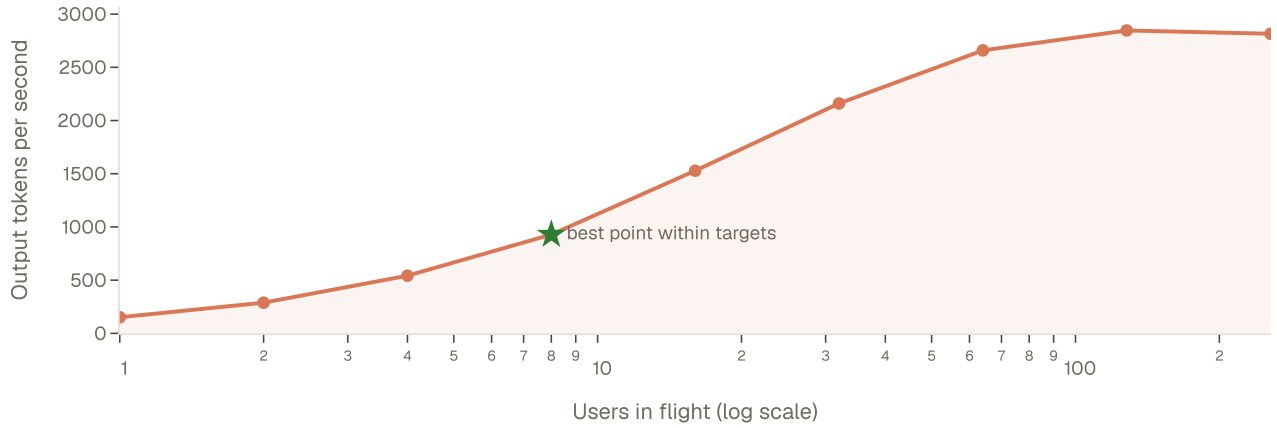
Each engine step, the scheduler fills a budget of tokens: first one token for every request already writing, then chunks of new prompts, while it has slots and cache to give. Whichever runs out first decides who waits.

Learn more: Lesson 35: Token budgets, max sequences and max batched tokens, How big should the budget be?
(https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/35-token-budgets-max-seqs-max-batched-tokens.md#3-how-big-should-the-budget-be)

Throughput against P99 time per token (labels: users in flight)



Throughput as users are added



Reading left to right: in the flat region throughput grows a lot while each user barely slows down. At the knee every extra user still adds throughput but slows everyone else noticeably. At the wall throughput stops growing and latency keeps climbing as requests wait.

WHAT WE MEASURED

kneebench kept a fixed number of requests in flight (1, 2, 4 and so on, doubling) and measured steady-state output tokens per second and per-token latency at each level.

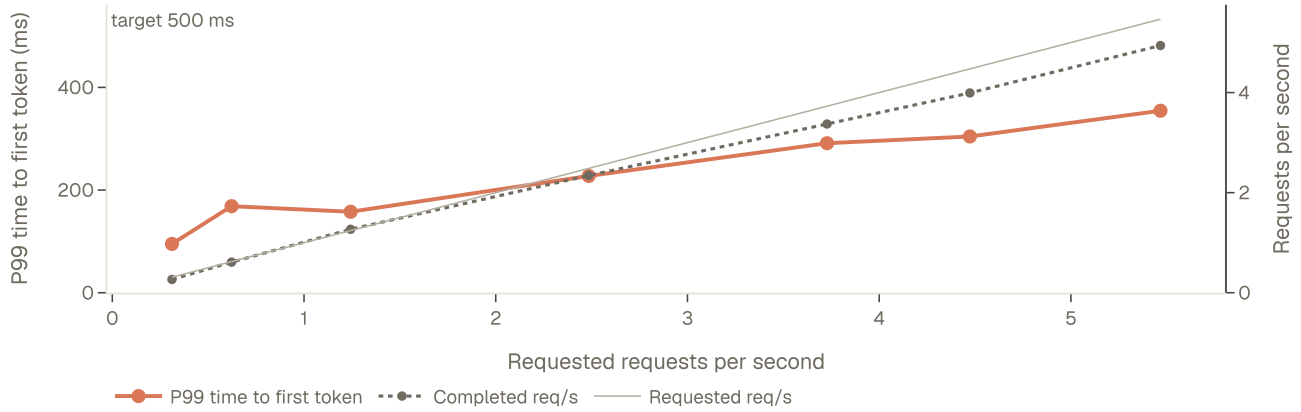
USERS IN FLIGHT	OUTPUT TOK/S	TPOT P50	TPOT TAIL	TTFT TAIL	KV CACHE PEAK	PREEMPTIONS	ERRORS
1	152	5.9 ms	6.3 ms	78 ms	0.5%	0	0%
2	288	6.2 ms	7.1 ms	133 ms	0.9%	0	0%
4	542	6.6 ms	6.8 ms	175 ms	1.8%	0	0%
8	928	7.7 ms	9.8 ms	500 ms	3.5%	0	0%
16	1,529	9.5 ms	10.3 ms	852 ms	7.3%	0	0%
32	2,161	13.3 ms	15.5 ms	2.01 s	15%	0	0%
64	2,659	21.9 ms	26.1 ms	4 s	29%	0	0%
128	2,846	40.9 ms	50.3 ms	8.53 s	58%	0	0%
256	2,815	80.5 ms	111 ms	17 s	100%	144	0%

WHY IT HAPPENS

Decoding shares one weight read across the whole batch, so small batches are nearly free. As the batch grows, each user's own cache reads and math start to dominate the step, and finally the server runs out of room and makes new requests wait.

Learn more: Lesson 25: Sweep concurrency and request rate, The one plot that matters (https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/25-sweep-concurrency-request-rate.md#3-the-one-plot-that-matters)

Random arrivals: first-token time as the rate rises (red: queue keeps growing)



Plan around this rate, not the concurrency number: it is what real, unsynchronized users can do to the server.

WHAT WE MEASURED

kneebench sent requests at fixed average rates with random (Poisson) gaps, from a quarter of the measured capacity to just past it, for 90 seconds each, then up to two more rates halfway between the best one that met your targets and the first that didn't, to pin the limit down. Time to first token bends up from about 2.48 requests per second. First-token times have the 277 ms network round trip from this location removed: an estimate of the server's own time.

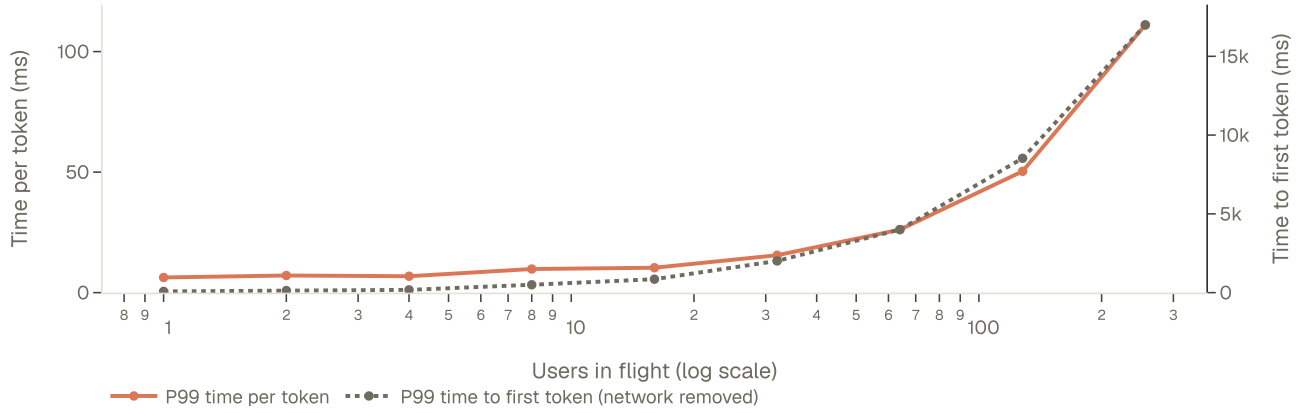
REQUESTED REQ/S	COMPLETED REQ/S	TTFT P50	TTFT TAIL	TPOT TAIL	TTFT GROWTH IN RUN
0.3	0.3	74.9 ms	95.1 ms	7.4 ms	0.9×
0.6	0.6	72.2 ms	169 ms	8.3 ms	1.1×
1.2	1.3	83.2 ms	158 ms	11.4 ms	1.3×
2.5	2.3	83.5 ms	227 ms	12.4 ms	1.0×
3.7	3.4	90.6 ms	291 ms	23.3 ms	1.0×
4.5	4.0	98.4 ms	304 ms	19.3 ms	1.0×
5.5	4.9	141 ms	355 ms	28.2 ms	0.9×

WHY IT HAPPENS

In the concurrency sweep a new request starts only when one finishes, so the test slows down with the server. Real users don't wait. Near capacity, random arrivals bunch up and briefly exceed what the server can start, so first-token time climbs before throughput is used up.

Learn more: Lesson 25: Sweep concurrency and request rate, Sweep 2: fixed request rate (https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/25-sweep-concurrency-request-rate.md#4-sweep-2-fixed-request-rate)

Latency tails as users are added (dashed: your targets)



This is where users start getting a worse experience than you promised. It usually comes before throughput saturates, which makes it your real limit.

WHAT WE MEASURED

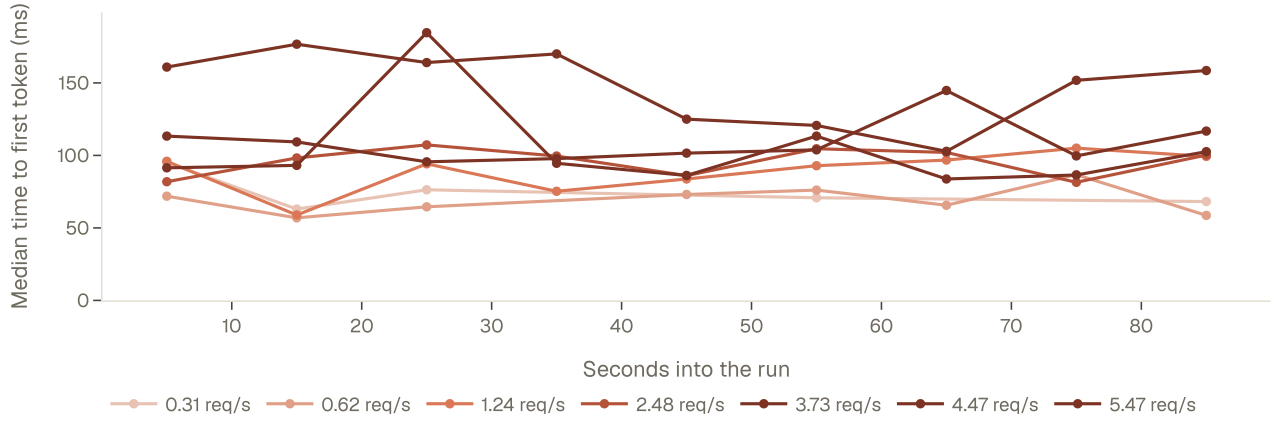
TPOT crosses its target at about 127 users; TTFT crosses its target at about 8 users. The crossing is interpolated between the last passing and first failing level. Time to first token is judged without the network: the 277 ms round trip from where kneebench ran, and any new-connection setup, is removed, because it doesn't change with load and your users add their own. What's left is an estimate of the server's own time.

WHY IT HAPPENS

Latency grows slowly at first, bends up past the knee, and crosses the target well before the GPU stops adding throughput. Capacity is the first crossing, not the plateau.

Learn more: Lesson 27: Find saturation, SLO failure and OOM boundaries, Wall 2: SLO failure (https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/27-find-saturation-slo-oom-boundaries.md#3-wall-2-slo-failure)

First-token time during each run (rising lines: past capacity)



This is the most this GPU can do for this kind of request. Past it, extra users aren't served faster, they wait.

WHAT WE MEASURED

Each doubling of users added less and less throughput; past 128 users it added under 5%. The server's waiting queue rose to 203 requests.

WHY IT HAPPENS

Batching gets cheaper per user until the per-user work dominates each step or memory runs out. After that, the server can't add work per step, so new requests queue instead.

Learn more: Lesson 27: Find saturation, SLO failure and OOM boundaries, Wall 1: saturation (https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/27-find-saturation-slo-oom-boundaries.md#2-wall-1-saturation)

Extra users mostly ride along on work the GPU does anyway, so the same hardware serves many more people with only a small slowdown for each.

WHAT WE MEASURED

One user at a time: 152 output tokens per second (5.9 ms per token). 8 users at once: 928 tokens per second in total (7.7 ms per token each).

WHY IT HAPPENS

With one user, each decode step is almost all reading the model's weights from memory. A second user reuses that same read; it only adds its own cache and a little math. The free lunch ends when those per-user extras add up, or when the KV cache *(the memory where the server keeps each request's context while it answers)* fills.

Learn more: Lesson 25: Sweep concurrency and request rate, The free lunch (https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/25-sweep-concurrency-request-rate.md#1-the-free-lunch)

Quoting the peak as capacity would promise users a speed they won't get. Plan with the operating point instead.

WHAT WE MEASURED

At 128 users the server produced the most tokens per second, but per-token latency there was 50.3 ms at P99.

WHY IT HAPPENS

The highest throughput sits at the wall, where requests already queue and latency is worst. Capacity is the most load that still meets the targets (goodput *(requests per second that met the latency targets)*), which is always to the left of the peak.

Learn more: Lesson 25: Sweep concurrency and request rate, How sweeps quietly go wrong (https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/25-sweep-concurrency-request-rate.md#6-how-sweeps-quietly-go-wrong)

That time is stolen from every other user in the batch.

WHAT WE MEASURED

An evicted request recomputes its prompt plus what it had written so far, about 2,250 tokens on average, at the prefill rate of this GPU.

WHY IT HAPPENS

vLLM's default response to a full cache is to discard an evicted request's cache and redo prefill later, so the work it had done is paid for twice.

Learn more: Lesson 27: Find saturation, SLO failure and OOM boundaries, Wall 3: memory, three faces (https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/27-find-saturation-slo-oom-boundaries.md#4-wall-3-memory-three-faces)

Real traffic arrives in bursts, and replicas fail. Planning right at the limit means every burst, or the loss of one replica, pushes users past your target.

WHAT WE MEASURED

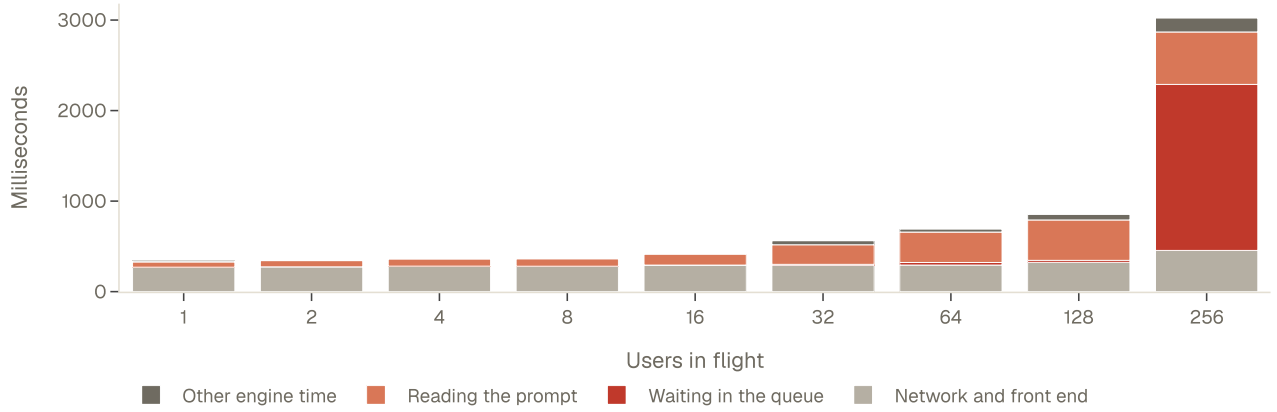
The highest random-arrival rate that met your targets, times 80%. If one replica may fail, the others must carry its share: with 2 replicas plan 1.79 per replica, with 4 replicas plan 2.68 per replica.

WHY IT HAPPENS

Near capacity, waiting time grows much faster than load: queues that are fine at 80% busy are several times longer at 90%. The margin absorbs bursts; the failure budget keeps the targets when the remaining replicas take over a lost one's traffic. 20% is a starting point; your own peak-to-average traffic is the better number.

Learn more: Lesson 43: Find the SLO-safe configuration and headroom, Bursts (https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/43-find-slo-safe-configuration-headroom.md#5-bursts)

Median time to first token, split by where it is spent



This says which part to fix: a large gap means distance or the front end, a growing queue means the server is admitting requests slower than they arrive, and a long prompt read means prefill itself.

WHAT WE MEASURED

Two stopwatches on the same requests: kneebench's, from sending to the first token, and vLLM's own, from when the request reached the engine. The gap is the network plus the server's front end (HTTP and tokenization). vLLM also splits its part into time waiting in the queue and time reading the prompt. Its percentiles are estimates from histogram buckets.

USERS	TTFT (CLIENT)	TTFT (ENGINE)	NETWORK AND FRONT END	QUEUE P50	QUEUE P99	PROMPT READ P50
1	341 ms	70.7 ms	270 ms	0.02 ms	0.02 ms	58.3 ms
2	347 ms	72.9 ms	274 ms	0.01 ms	0.01 ms	68.5 ms
4	369 ms	87.4 ms	281 ms	0.01 ms	0.01 ms	79.8 ms
8	366 ms	85.9 ms	281 ms	0.00 ms	6.9 ms	82.5 ms
16	412 ms	119 ms	293 ms	0.00 ms	83.2 ms	121 ms
32	562 ms	272 ms	290 ms	11 ms	1.03 s	217 ms
64	690 ms	397 ms	294 ms	28.4 ms	3.59 s	336 ms
128	853 ms	529 ms	323 ms	22.2 ms	8.23 s	446 ms
256	3.02 s	2.57 s	455 ms	1.83 s	15.4 s	579 ms

WHY IT HAPPENS

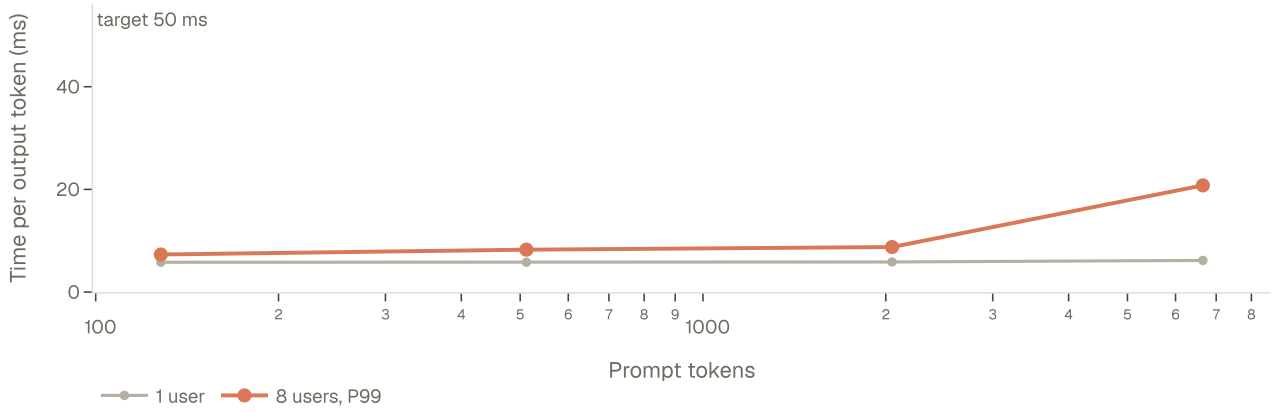
Time to first token is a sum. The client sees all of it; the engine only sees what happens after the request reaches it. Comparing the two is the only way to tell from outside whether a slow first token is the server's fault.

Learn more: Lesson 30: Read engine metrics and request state, Two stopwatches on the same request

(https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/30-read-engine-metrics-request-state.md#6-two-stopwatches-on-the-same-request)

How request size changes things

Time per output token as prompts grow



Users with short prompts stream slower when they share the server with long-prompt traffic. Plan latency for your longest common prompts, not the average.

WHAT WE MEASURED

The same prompt-length sweep, repeated at the operating point the concurrency sweep found, with the output fixed.

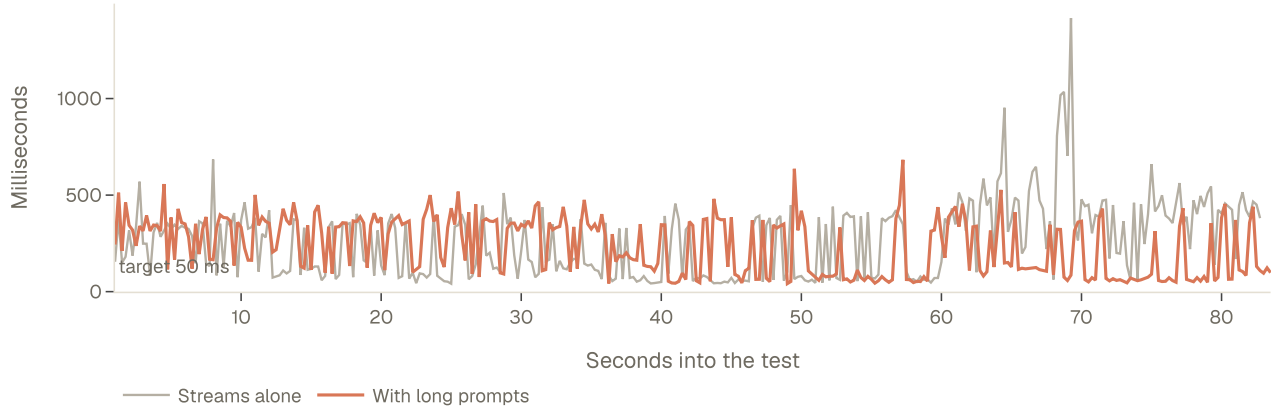
PROMPT TOKENS	TPOT P50	TPOT TAIL	PEAK CACHE
128	6.1 ms	7.3 ms	0.5%
512	6.5 ms	8.3 ms	1.0%
2,048	8 ms	8.8 ms	3.3%
6,656	13.8 ms	20.8 ms	10%

WHY IT HAPPENS

Each step of decode *(the word-by-word step that writes the answer, limited by GPU memory speed)* rereads the KV cache of every earlier token, for every request in the batch. Longer prompts mean more cache to reread, so each step, and every user in it, gets slower.

Learn more: Lesson 26: Sweep prompt and output length, Sweep the prompt (https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/26-sweep-prompt-output-length.md#2-sweep-the-prompt-output-fixed-at-256)

Longest gap between streamed chunks; dotted lines are long prompts arriving



Pauses this large without long prompts come from between kneebench and the engine (the network, a gateway bundling chunks, or a busy client), not from the server's scheduling. Run the test next to the server to measure the real effect.

Run kneebench next to the server, on a machine that keeps up

Expected Without network jitter, any pause the long prompts cause stands out.

Risk None.

Verify Rerun; the streams alone should pause no more than a few normal gaps.

WHAT WE MEASURED

32 users streamed long answers while a 6,912-token prompt arrived 4 times, 10 seconds apart; the same streams ran once without them. kneebench timed every gap between streamed chunks. 3 of 4 long prompts were followed within 1.5 s by a pause, which is what ties the pauses to them. The long prompts' own first token took 1.45 s. The median barely moves and P99 can stay normal: with a few spikes among thousands of gaps, only the longest pause and P99.9 show them.

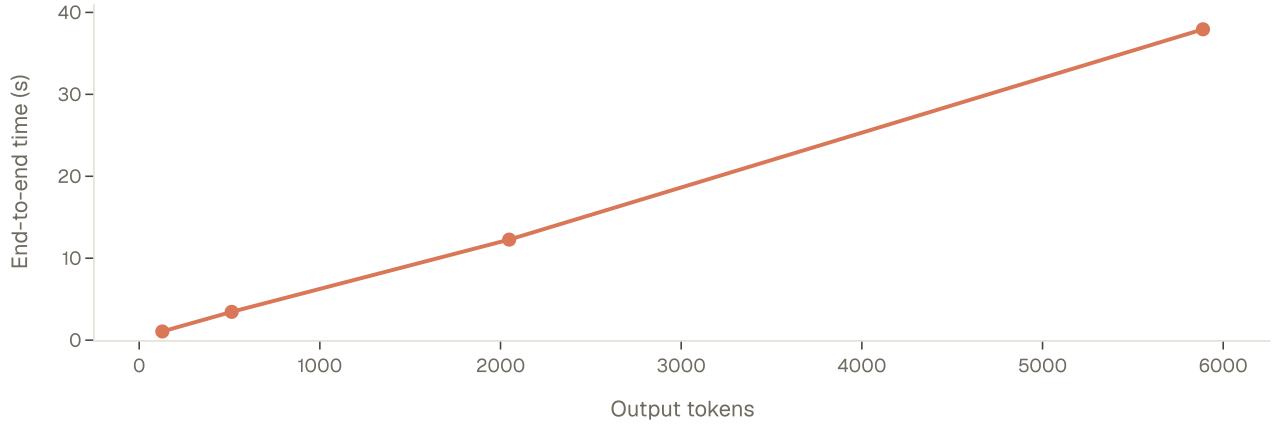
RUN	GAP P50	GAP P99	GAP P99.9	LONGEST PAUSE	STALLS
Streams alone	10.1 ms	105 ms	404 ms	1.42 s	9,395
With long prompts	11.5 ms	82.7 ms	354 ms	683 ms	5,946

WHY IT HAPPENS

While a prompt is read (prefill), the engine step carrying it is much longer than a normal step that writes one token per user, and every user streaming at that moment waits for it. vLLM reads long prompts in chunks of at most max_num_batched_tokens (about 8,192 here) per step, so that budget sets how long the pause can be.

Learn more: Lesson 44: Reproduce the long-prompt interference problem, Read the results (https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/44-reproduce-long-prompt-interference.md#4-read-the-results)

One user: total answer time as the output grows



Long answers are slow answers: reasoning models and long-form writing pay this per token, so cap max_tokens where your product allows and stream the output to users.

WHAT WE MEASURED

One user at a time, answers from 128 to 5,888 tokens with 1,024-token prompts. The line starts at about -68.34 ms (prompt processing plus the network) and its slope is the time per output token.

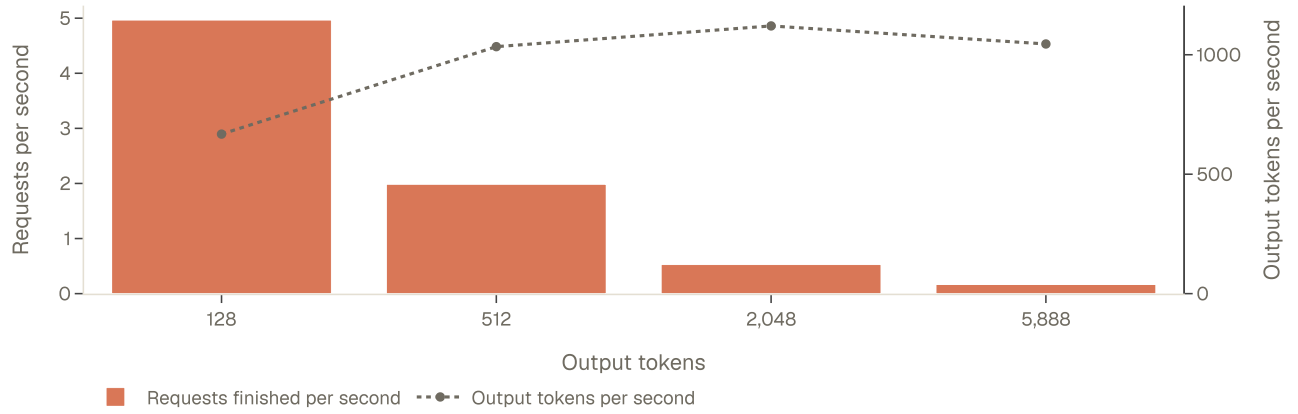
OUTPUT TOKENS	E2E P50	TTFT P50	TPOT P50
128	1.06 s	30.7 ms	5.8 ms
512	3.45 s	34.7 ms	5.9 ms
2,048	12.3 s	28.5 ms	5.9 ms
5,888	37.9 s	40 ms	6 ms

WHY IT HAPPENS

The answer is written one token per step, called decode, and each step reads the whole model from memory. Output tokens are the expensive ones; the prompt is read once.

Learn more: Lesson 26: Sweep prompt and output length, Sweep the output (https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/26-sweep-prompt-output-length.md#3-sweep-the-output-prompt-fixed-at-1000)

8 users: longer answers, fewer finished requests



Tokens per second can look healthy while far fewer requests finish. Size for requests per second at your real answer length.

WHAT WE MEASURED

The output-length sweep repeated at the operating point, prompts fixed.

OUTPUT TOKENS	REQUESTS/S	OUTPUT TOKENS/S	PEAK CACHE	EVICTIONS
128	5.0	668	1.6%	0
512	2.0	1,034	2.2%	0
2,048	0.5	1,121	4.1%	0
5,888	0.2	1,045	9.1%	0

WHY IT HAPPENS

A request keeps its KV cache until its last token. Requests that decode for a minute hold memory for a minute, and requests that start small can grow into each other later, which forces preemption *(the server evicting a running request when memory runs out, then recomputing it later)*.

Learn more: Lesson 26: Sweep prompt and output length, Shape decides how many fit (https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/26-sweep-prompt-output-length.md#4-shape-decides-how-many-fit)

Shortening answers saves far more time than shortening prompts. Prompts aren't free, though: they still cost first-token time and cache.

WHAT WE MEASURED

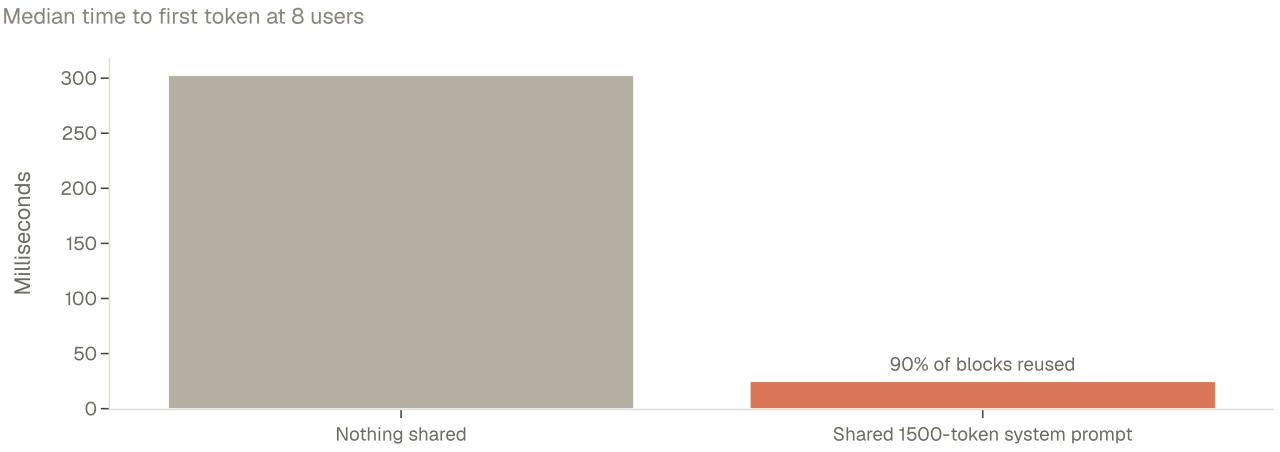
Time per output token for one user (5.8 ms) divided by the cost of one more prompt token (19 µs, from the prompt-length sweep).

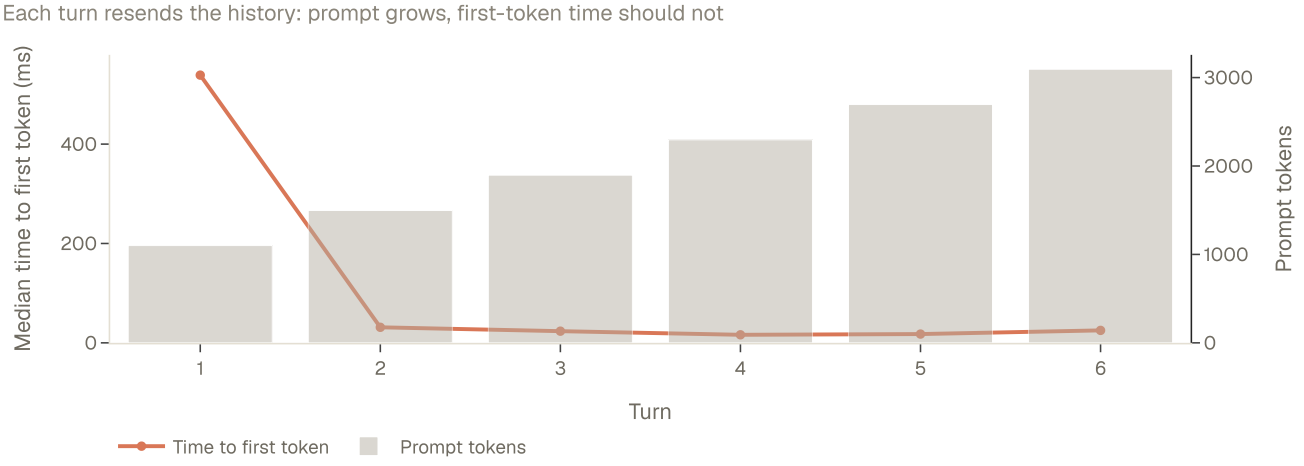
WHY IT HAPPENS

The prompt is read in parallel, called prefill, limited by compute. The answer is written one token per step, called decode, each step reading the whole model from memory.

Learn more: Lesson 26: Sweep prompt and output length, Two lengths, two machines (https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/26-sweep-prompt-output-length.md#1-two-lengths-two-machines)

What prompt reuse buys





The cache only reuses a prompt up to the first token that differs. Ordered by score, two requests with the same passages match only until their first passages differ. Sorting retrieved passages by document id before building the prompt lets the server reuse them. Keep the question last for the same reason.

WHAT WE MEASURED

8 users sent retrieval prompts: 800 tokens of instructions, the same 5 passages of 300 tokens, then a new question each time. Once with the passages shuffled per request, as a retriever's scores would order them, once sorted by document number.

WHY IT HAPPENS

A passage's cached state was computed while reading everything before it, so it is only valid after exactly the same text. Changing the order changes what comes before.

Learn more: Lesson 48: Shared prompts, RAG context and multi-turn reuse, Shape 3: retrieval, the hard case

(https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/48-shared-prompts-rag-multi-turn-reuse.md#5-shape-3-retrieval-the-hard-case)

Every prompt version, tenant variant or A/B arm is its own cache entry. Below this count they all stay cached; above it the server keeps evicting the one it needs next. Fewer variants, or routing each variant to the same replica, keeps reuse high.

WHAT WE MEASURED

Not a test: a kneebench estimate. The server's 542,400-token cache, minus what 8 running requests need, divided by one 1,500-token prompt in whole cache blocks.

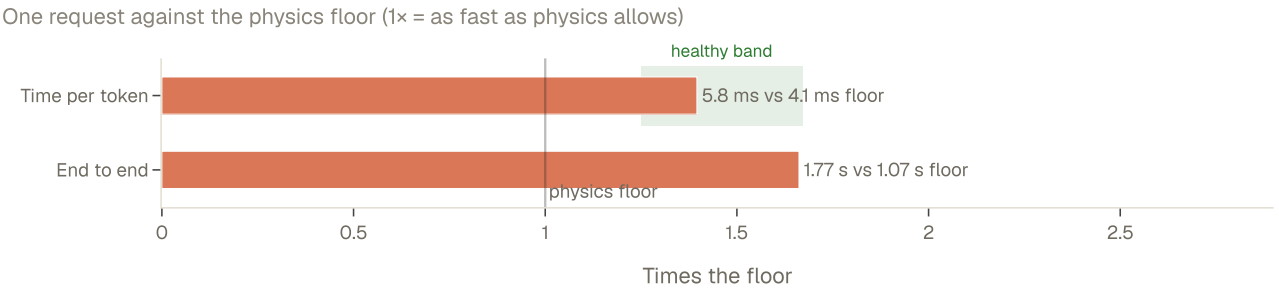
WHY IT HAPPENS

The cache evicts the least recently used blocks when it fills. When prompts come back in a cycle, that is always the one needed next, so reuse falls to near zero as soon as the set no longer fits, not gradually.

Learn more: Lesson 49: RadixAttention, eviction and cache churn, Cache churn (https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/49-radixattention-eviction-cache-churn.md#6-cache-churn)

What the hardware allows

Background on what this hardware can do. Open a card for the explanation.



The engine is doing its job: there is nothing to fix for single-request speed. The sweeps measure how this changes as users are added.

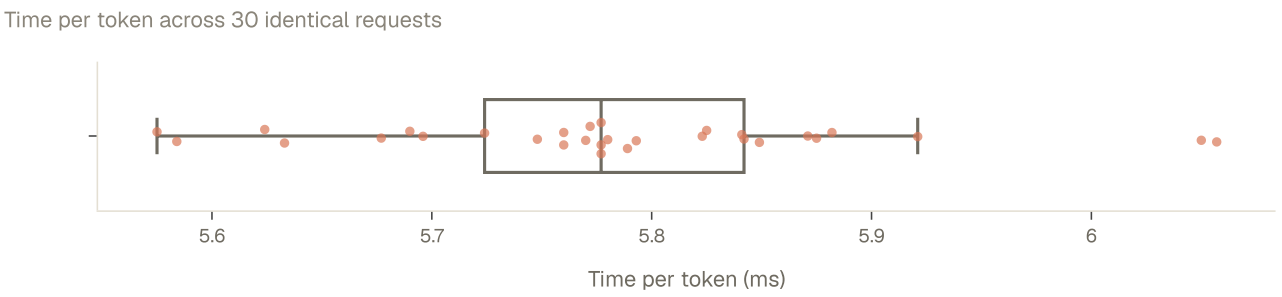
WHAT WE MEASURED

With nothing else running, the median TPOT *(time per output token, the gap between words once the answer is streaming)* across 30 identical requests was 5.8 ms. The floor for this model on NVIDIA H100 NVL 94GB is 4.1 ms, so decoding reaches 72% MBU *(model bandwidth utilization, how much of the GPU's memory speed decoding actually uses)*.

WHY IT HAPPENS

With one request, each decode step is almost entirely reading weights from memory, so speed is bounded by bandwidth. Well-tuned engines reach 60 to 80% of it; Databricks reported about 60% at batch 1 on two H100s. The gap is launch overhead and small operations that can't use the full bandwidth. *Like a car on an empty highway: if it can't get near the speed limit, the problem is the car, not the traffic.*

Learn more: Lesson 24: Establish the single-request baseline, Compare against the floors (https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/24-establish-single-request-baseline.md#4-compare-against-the-floors)



The measurement is repeatable, so differences in later runs are real.

WHAT WE MEASURED

30 identical requests: median 5.8 ms, P99 *(99 out of 100 requests were at least this fast; the slow tail users notice)* 6.1 ms, coefficient of variation 1.9%.

WHY IT HAPPENS

Every baseline request is identical, so they should take the same time. Spread means something else is involved: another workload on the GPU, throttling, or a noisy network path.

Learn more: Lesson 24: Establish the single-request baseline, When it looks off (https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/24-establish-single-request-baseline.md#5-when-it-looks-off)

Short prompts leave most of the GPU's math units idle; that is expected, not a problem. Long prompts should keep them much busier.

WHAT WE MEASURED

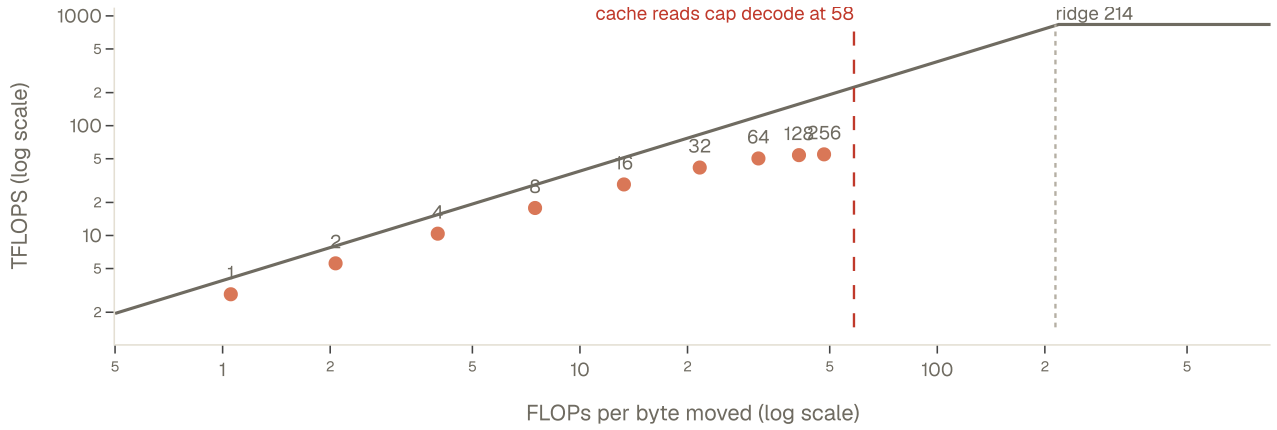
From the prompt-length sweep, each extra prompt token costs 19 µs, against 19 µs if every math unit were busy: 102% MFU *(model FLOPs utilization, how much of the GPU's math throughput prefill actually uses)*. Measured as a slope, so the network round trip and fixed overheads drop out.

WHY IT HAPPENS

A few hundred tokens isn't enough work to keep every math unit busy, so fixed overheads dominate. Past a few thousand tokens prefill becomes compute-bound and its cost per token is close to the math floor; real kernels usually reach well under the datasheet peak.

Learn more: Lesson 59: Profile prefill vs decode, The prefill profile (https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/59-profile-prefill-vs-decode.md#3-the-prefill-profile)

Decode steps against the GPU's roofline (labels: users)



Left of the ridge, the GPU waits on memory: a faster chip's math changes nothing, and fewer bytes (smaller weights, an FP8 cache) or more users per step help. Each step reads the weights once for everyone, so adding users is nearly free until every request's cache reads add up. At 256 users the server reached 29% of what the roofline allows.

WHAT WE MEASURED

For each load level: the math and the bytes of one decode step at that many users, with 2,250-token contexts (the prompt plus half the answer), against the measured time per token. The NVIDIA H100 NVL 94GB peaks are 835 TFLOPS (BF16) and 3.90 TB/s.

USERS	FLOPS PER BYTE	ACHIEVED TFLOPS	ROOFLINE TFLOPS	OF THE LINE
1	1	2.9	4.1	71%
2	2	5.6	8.1	69%
4	4	10.4	15.6	67%
8	7	17.8	29.2	61%
16	13	29.2	51.8	56%
32	22	41.6	84.4	49%
64	32	50.3	123.2	41%
128	41	53.9	159.9	34%
256	48	54.8	188.0	29%

WHY IT HAPPENS

A step takes about as long as the larger of its math and its memory traffic. Intensity, the math per byte, decides which: below the ridge (peak math divided by bandwidth) memory wins. Real roofs sit about 20% below the datasheet, so 70-80% of the line is good.

Learn more: Lesson 61: Compute-bound vs memory-bound and the roofline model, Why this picture runs inference

(https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/61-compute-vs-memory-bound-roofline.md#4-why-this-picture-runs-inference)

This is the ruler every later number is measured against. Real servers always land above it; how far above tells you whether something is misconfigured.

WHAT WE MEASURED

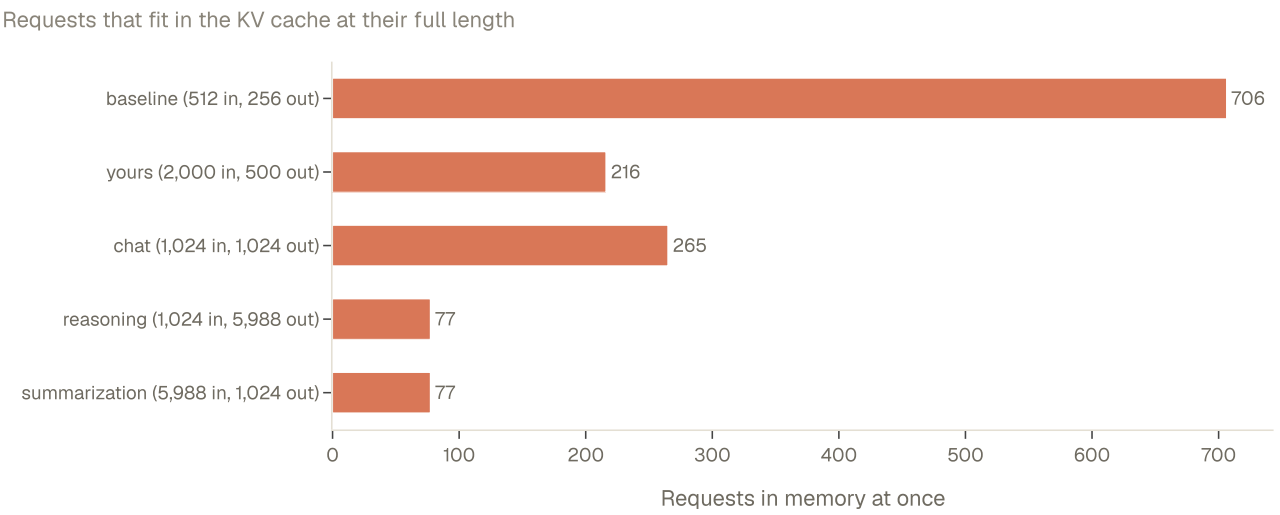
Before sending any traffic, kneebench worked out the floor *(the fastest time physics allows on this hardware; nothing can beat it)* from the model's size and the published speeds of NVIDIA H100 NVL 94GB. Reading all 16.1 GB of weights, plus the cache of the conversation so far, takes 4.1 ms at 3.90 TB/s, so no token can arrive faster. Reading a 512-token prompt needs at least 9.9 ms of math at 835 TFLOPS.

SHAPE	PROMPT TOKENS	OUTPUT TOKENS	TTFT FLOOR	TPOT FLOOR	END-TO-END FLOOR
baseline	512	256	9.9 ms	4.1 ms	1.07 s
yours	2,000	500	39.7 ms	4.2 ms	2.13 s
chat	1,024	1,024	20 ms	4.2 ms	4.29 s
reasoning	1,024	5,988	20 ms	4.3 ms	25.5 s
summarization	5,988	1,024	126 ms	4.3 ms	4.56 s

WHY IT HAPPENS

Writing the answer is called decode: each step reads every weight and the whole KV cache so far from GPU memory, so time per token is at least those bytes divided by memory bandwidth; long conversations make every token slower. Reading the prompt is called prefill: about two math operations per parameter per prompt token, plus attention, which grows with the square of the prompt, so it is bounded by the GPU's math throughput. End to end is the first token plus one decode step for each remaining token.

Learn more: Lesson 59: Profile prefill vs decode, Napkin math first (https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/59-profile-prefill-vs-decode.md#2-napkin-math-first)



This is the memory ceiling on how many of these requests can be in flight together. Past it, the server either queues new requests or evicts running ones.

WHAT WE MEASURED

The cache size divided by each shape's final length (prompt plus output), rounded up to whole 16-token blocks, since the server hands out memory a block at a time.

SHAPE	PROMPT TOKENS	OUTPUT TOKENS	REQUESTS THAT FIT
baseline	512	256	706
yours	2,000	500	216
chat	1,024	1,024	265
reasoning	1,024	5,988	77
summarization	5,988	1,024	77

WHY IT HAPPENS

A request holds memory for its prompt plus everything it has generated so far, so its final length is what counts, not its starting length. Long answers start small and grow, which is why reasoning workloads run out of memory later than they first appear to.

Learn more: Lesson 26: Sweep prompt and output length, Shape decides how many fit (https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/26-sweep-prompt-output-length.md#4-shape-decides-how-many-fit)

Every request in flight holds this much memory per token of its prompt and answer. The pool is the total budget all users share.

WHAT WE MEASURED

From the model's architecture: 128 KiB per token. From the server's /metrics: 542,400 tokens of cache after reserving 90% of GPU memory and loading the weights.

WHY IT HAPPENS

For every token in a conversation, each layer keeps a key and a value vector so later tokens can look back at it: bytes per token = 2 x layers x KV heads x head size x bytes per number. vLLM reserves a share of GPU memory, loads the weights, and gives what's left to this cache. Every decode step rereads the cache of every token so far, which is why long prompts also slow decode.

Learn more: Lesson 26: Sweep prompt and output length, The prompt changes decode too (https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/26-sweep-prompt-output-length.md#2-sweep-the-prompt-output-fixed-at-256)

kneebench shortened the longer side of these shapes. Their results describe the adjusted lengths, not the originals.

Raise the context limit if your real traffic is longer

```
--max-model-len
```

Expected The original shapes can be measured as-is.

Risk A longer limit needs more cache per request; check the memory findings after.

Verify Rerun; this note disappears.

WHAT WE MEASURED

Prompt plus output plus room for the chat template must fit in 8,192 tokens, with 15% to spare: sweeps vary each request's length a little, and the longest ones must still fit.

WHY IT HAPPENS

A request can't be longer than the server's context window, so a shape that doesn't fit would fail every request instead of measuring anything.

Learn more: Lesson 26: Sweep prompt and output length, Running the sweep (https://github.com/S1LV3RJ1NX/inference-engineering-notes/blob/main/notes/26-sweep-prompt-output-length.md#5-running-the-sweep)

Checks that passed

15 checks passed show them

GOOD **Does the server stop when a user leaves?** A01
kneebench hung up on 16 requests 1 s after sending them and the server stopped: 150 tokens each instead of 1,000.

GOOD **Time to first token versus its floor** B03
The server's own metrics put time to first token at about 24.3 ms for a 512-token prompt (2.4× its 9.9 ms floor): normal for a short prompt.

GOOD **Did the GPU change during the run?** B06
Time per token at the end of the run matched the start (5.8 ms and 5.8 ms).

GOOD **The best load that still meets your targets** C03
The best operating point is 8 users in flight: 928 output tokens per second with every target met at P99.

GOOD **Did each load level reach a steady state?** H01
All 9 load levels ran long enough at full load to measure throughput.

GOOD **Did the client keep up?** H02
The load generator held the requested concurrency at every level (9 checked).

GOOD **Did every answer reach the requested length?** H04
Every point generated the full requested output length (46 checked).

GOOD **Do token counts match the server?** H05
Token counts match the server's, apart from 35 tokens the server adds to every prompt for the chat template.

GOOD **Versions under test** H07
vLLM 0.29.0 is within the tested range (0.29.0 up to 0.33.0). Recorded: kneebench 0.1.0.dev0, AIPerf 0.13.0.

GOOD **Is something between kneebench and the server limiting load?** H09
No sign of a gateway or proxy limiting the load (9 levels checked).

GOOD **Was kneebench alone on the server?** H10
The server was idle when the test started, and never ran more than one request while kneebench sent one at a time.

GOOD **Room for a full-length request** T04
The cache holds 66× full-length (8,192-token) requests: room to spare.

GOOD **Measured speed versus the claimed GPU** T06
Measured 5.8 ms per token is consistent with NVIDIA H100 NVL 94GB.

GOOD **Memory the server reports versus the claimed GPU** T07
Weights (15.0 GiB) plus the reported cache (66.2 GiB) fit in NVIDIA H100 NVL 94GB at 90% utilization.

GOOD **Errors and timeouts under load** W07
No requests failed up to 256 users.

What this test can't see

kneebench only sees the endpoint. It has no GPU telemetry; memory and queue signals come from vLLM's /metrics.

Prompts are random text, so they never share a beginning. Real traffic often repeats system prompts, which prefix caching can speed up; your real time to first token may be lower.

Results describe this server, with these flags, at this time. Change one setting at a time and rerun to compare.

This test ran 279 ms (one network round trip) away from the server. Time per token is reliable from here; first-token times include the network.

From here the network adds as much as the server takes, and its jitter lands in the slow tail. So each first-token percentile is the lower of two estimates: the client's with the round trip removed, and the server's own histogram (whose percentiles are rounded to its buckets).

The server has prefix caching *(reusing work for prompts that start with the same text)* on; this test's prompts hit it 4.8% of the time. Real traffic that repeats system prompts will likely start answers faster than measured here.

Glossary

12 terms used in this report

KV cache the memory where the server keeps each request's context while it answers

MBU model bandwidth utilization, how much of the GPU's memory speed decoding actually uses

MFU model FLOPs utilization, how much of the GPU's math throughput prefill actually uses

P99 99 out of 100 requests were at least this fast; the slow tail users notice

TPOT time per output token, the gap between words once the answer is streaming

TTFT time to first token, how long a user waits before the answer starts appearing

decode the word-by-word step that writes the answer, limited by GPU memory speed

floor the fastest time physics allows on this hardware; nothing can beat it

goodput requests per second that met the latency targets

preemption the server evicting a running request when memory runs out, then recomputing it later

prefill the step that reads the whole prompt at once, limited by GPU math

prefix caching reusing work for prompts that start with the same text

Appendix: raw numbers

Every number behind this report as plain tables; the JSON and Markdown exports carry the same data

MEASURED POINTS

#	PHASE	STATUS	PROMPT TOKENS	OUTPUT TOKENS	LOAD	REQUESTS	TTFT P50	TTFT P99	TPOT P50	TPOT P99	E2E P50	OUTPUT TOK/S	ERROR %
0	baseline	done	512	256	1	30	304 ms	347 ms	5.8 ms	6.1 ms	1.77 s	143.2	0.0
1	concurrency	done	2,000	500	1	16	341 ms	591 ms	5.9 ms	6.3 ms	3.34 s	150.9	0.0
2	concurrency	done	2,000	500	2	20	347 ms	410 ms	6.2 ms	7.1 ms	3.46 s	284.4	0.0
3	concurrency	done	2,000	500	4	40	369 ms	531 ms	6.6 ms	6.8 ms	3.64 s	533.9	0.0
4	concurrency	done	2,000	500	8	80	366 ms	776 ms	7.7 ms	9.8 ms	4.26 s	910.8	0.0
5	concurrency	done	2,000	500	16	144	412 ms	1.3 s	9.5 ms	10.3 ms	5.18 s	1,510.0	0.0
6	concurrency	done	2,000	500	32	224	562 ms	2.55 s	13.3 ms	15.5 ms	7.26 s	2,161.9	0.0
7	concurrency	done	2,000	500	64	384	690 ms	4.93 s	21.9 ms	26.1 ms	11.8 s	2,659.2	0.0
8	concurrency	done	2,000	500	128	512	853 ms	9.52 s	40.9 ms	50.3 ms	21.4 s	2,924.9	0.0
9	concurrency	done	2,000	500	256	1,024	3.02 s	19.6 s	80.5 ms	111 ms	43.6 s	2,823.3	0.0
10	rate	done	2,000	500	1.24 req/s	117	373 ms	1.6 s	7.4 ms	11.4 ms	4.11 s	632.9	0.0
11	rate	done	2,000	500	2.48 req/s	220	375 ms	1.57 s	9.1 ms	12.4 ms	4.97 s	1,170.0	0.0
12	rate	done	2,000	500	3.73 req/s	316	379 ms	2.47 s	11.6 ms	23.3 ms	6.26 s	1,679.4	0.0
13	rate	done	2,000	500	4.47 req/s	379	383 ms	2.2 s	14 ms	19.3 ms	7.41 s	1,992.7	0.0
14	rate	done	2,000	500	5.47 req/s	471	428 ms	1.27 s	22.2 ms	28.2 ms	11.3 s	2,463.2	0.0
15	rate	done	2,000	500	0.62 req/s	55	349 ms	1.14 s	6.5 ms	8.3 ms	3.57 s	302.4	0.0
16	rate	done	2,000	500	0.31 req/s	24	352 ms	807 ms	6.2 ms	7.4 ms	3.39 s	127.6	0.0
17	rate	done	2,000	500	3.73 req/s	339	404 ms	1.15 s	11.7 ms	18.7 ms	6.27 s	1,803.3	0.0
18	rate	done	2,000	500	4.47 req/s	396	511 ms	9.75 s	16.6 ms	47.1 ms	9.38 s	2,094.5	0.5

#	PHASE	STATUS	PROMPT TOKENS	OUTPUT TOKENS	LOAD	REQUESTS	TTFT P50	TTFT P99	TPOT P50	TPOT P99	E2E P50	OUTPUT TOK/S	ERROR %
19	shape_prompt	done	128	256	1	6	293 ms	299 ms	5.8 ms	6 ms	1.81 s	144.1	0.0
20	shape_prompt	done	128	256	8	32	285 ms	341 ms	6.1 ms	7.3 ms	1.81 s	1,055.8	0.0
21	shape_prompt	done	512	256	1	6	303 ms	323 ms	5.8 ms	5.9 ms	1.85 s	143.5	0.0
22	shape_prompt	done	512	256	8	32	321 ms	528 ms	6.5 ms	8.3 ms	1.94 s	955.3	0.0
23	shape_prompt	done	2,048	256	1	6	327 ms	336 ms	5.9 ms	6 ms	1.87 s	140.9	0.0
24	shape_prompt	done	2,048	256	8	32	362 ms	792 ms	8 ms	8.8 ms	2.41 s	812.8	0.0
25	shape_prompt	done	6,656	256	1	6	503 ms	546 ms	6.2 ms	6.9 ms	2.12 s	122.9	0.0
26	shape_prompt	done	6,656	256	8	32	808 ms	2.27 s	13.8 ms	20.8 ms	4.32 s	438.4	0.0
27	shape_output	done	1,024	128	1	6	307 ms	311 ms	5.8 ms	5.9 ms	1.06 s	123.5	0.0
28	shape_output	done	1,024	128	8	32	348 ms	542 ms	8.1 ms	11.4 ms	1.37 s	667.4	0.0
29	shape_output	done	1,024	512	1	6	311 ms	348 ms	5.9 ms	7.3 ms	3.45 s	148.3	0.0
30	shape_output	done	1,024	512	8	32	332 ms	548 ms	6.8 ms	7.4 ms	3.8 s	1,031.2	0.0
31	shape_output	done	1,024	2,048	1	3	305 ms	327 ms	5.9 ms	5.9 ms	12.3 s	164.9	0.0
32	shape_output	done	1,024	2,048	8	32	334 ms	526 ms	6.9 ms	7.3 ms	14.3 s	1,093.7	0.0
33	shape_output	done	1,024	5,888	1	3	320 ms	336 ms	6 ms	11.5 ms	37.9 s	128.7	0.0
34	shape_output	done	1,024	5,888	8	24	342 ms	552 ms	7.6 ms	7.8 ms	44.6 s	989.3	0.0
35	shape_map	done	1,024	1,024	8	32	333 ms	544 ms	6.7 ms	6.9 ms	7.1 s	1,100.6	0.0
36	shape_map	done	1,024	5,988	8	24	353 ms	540 ms	7.7 ms	7.8 ms	45.4 s	987.4	0.0
37	shape_map	done	5,988	1,024	8	32	567 ms	2.07 s	9.7 ms	10.4 ms	10.5 s	752.5	0.0
38	interference_base	done	128	6,884	scheduled	32	968 ms	1.26 s	12 ms	13.4 ms	83 s	2,583.5	0.0
39	interference	done	128	6,884	scheduled	36	865 ms	1.51 s	12.1 ms	19.9 ms	83.8 s	2,577.0	0.0

#	PHASE	STATUS	PROMPT TOKENS	OUTPUT TOKENS	LOAD	REQUESTS	TTFT P50	TTFT P99	TPOT P50	TPOT P99	E2E P50	OUTPUT TOK/S	ERROR %
40	reuse_control	done	1,600	128	8	32	620 ms	1.2 s	6.8 ms	9.4 ms	1.49 s	629.9	0.0
41	reuse_shared	done	100	128	8	32	300 ms	981 ms	6.3 ms	6.9 ms	1.1 s	807.4	0.0
42	reuse_turns	done	100	300	8	96	299 ms	859 ms	6.7 ms	8.1 ms	2.3 s	1,011.7	0.0
43	rag_ranked	done	2,350	128	8	32	469 ms	1.79 s	6.8 ms	8.9 ms	1.37 s	613.6	0.0
44	rag_sorted	done	2,350	128	8	32	306 ms	928 ms	6.3 ms	12.2 ms	1.41 s	717.1	0.0
45	cancel	done	500	1,000	8	n/a	n/a	n/a	n/a	n/a	n/a	n/a	n/a
46	baseline_end	done	512	256	1	30	299 ms	341 ms	5.8 ms	6.6 ms	1.77 s	142.9	0.0

CLIENT DISTANCE

DNS	TCP CONNECT	TLS HANDSHAKE	HTTP ROUND TRIP (/HEALTH)
1.7 ms	289 ms	294 ms	279 ms

SETUP

ITEM	VALUE
Model	unsloth/Meta-Llama-3.1-8B-Instruct
Parameters	8,030,261,248
Weight bytes	16,060,522,496
Weight dtype	bfloat16
KV bytes per token	131,072
GPU	1× NVIDIA H100 NVL 94GB
GPU memory (GiB)	93.6
GPU bandwidth (TB/s)	3.90
KV cache pool (tokens)	542,400
max_model_len	8,192
gpu_memory_utilization	0.90
Default max_num_seqs for this vLLM and GPU	1,024
vLLM	0.29.0
AIPerf	0.13.0